

VQE senza termine DM — trimero ad anello

Note di lavoro — Tesi triennale, Samuele Schiavi

Relatore: Prof. Alessandro Chiesa

Introduzione

Questo documento raccoglie i file della fase VQE **senza** termine DM per il trimero ad anello — mirror diretto, per $N = 3$, di quanto fatto per il dimero (`vqe_dimer.py`). Un ansatz PMA con branching e blocco RBS (produzione principale), lo stesso con blocco W (mirror dell'ansatz "originale" del dimero), un ansatz PMA senza branching, due notebook "spiegato" (guide interattive).

Scope. Tutti i file qui descritti evitano deliberatamente il termine DM. Per l'anello esiste già del materiale di VQE con DM (`analisi_espressivita_PMA_anello.ipynb`, e `confronto_ansatz_entangler_trimer_anello.ipynb`, che estende questi stessi ansatz aggiungendo lo sweep con DM Opzione B): non rientra in questo documento, e le scelte qui descritte non dipendono da esso. Per i moduli `.py`, oltre a descrizione/verifiche/conclusione, sono elencate anche le funzioni interne e come si usa il file.

`vqe_trimer_ring.py` + `vqe_trimer_ring.png`

Modulo di produzione principale: ansatz HA (hardware-efficient, 9 parametri) e PMA con blocco RBS e branching esplicito (stato iniziale scelto fra i multipletti di Kambe a seconda che b sia sopra o sotto $b_c(J, J')$), 6 parametri a $K = 1$. Metodologia di ottimizzazione a due stadi (multistart $R = 6$, COBYLA + polish L-BFGS-B) per mandato del relatore.

Verifiche. Self-test (ansatz a parametri nulli riproduce lo stato di Kambe atteso, errore 1.1×10^{-15}) e script principale completo rieseguiti in questa sessione: fidelity $\mathcal{F} = 1$ esatto su tutti i 10 punti della griglia standard, per entrambi gli ansatz. Figura rigenerata e confrontata byte per byte con quella già in possesso: identica.

Conclusione. Modulo maturo. Il vantaggio del PMA (6 parametri) su HA (9 parametri) a parità di fidelity esatta è la stessa lezione già vista nel dimero, confermata su un sistema frustrato a 3 corpi.

Funzioni interne.

- `make_ha_ansatz(reps)` — l'ansatz hardware-efficient generico ($R_y + CZ$, $3(\text{reps}+1)$ parametri).
- `rbs_block(qc, phi, q0, q1)` — il blocco RBS (rotazione di Givens reale), stessa implementazione validata per $N = 2$.
- `_select_target_block(J, Jp, b)` — sceglie quale blocco di Kambe (e quale M) è il fondamentale a questo (J, J', b) : B o C sotto b_c (a seconda del segno/rapporto di J, J'), A sopra.
- `_prepare_initial_state(qc, block, M)` — il circuito che prepara lo stato di Kambe scelto: banale (solo X) per C e per A estremo, generico (`prepare_state`) per B e A intermedio.
- `make_pma_ansatz(J, Jp, b, K)` — l'ansatz PMA completo: stato iniziale + K layer di RBS sui tre bond + R_y indipendenti finali.

- `_energy(params, ansatz, hamiltonian, estimator)` — la funzione obiettivo per `scipy.optimize.minimize` (valore atteso dell'energia).
- `run_vqe(J, Jp, b, ansatz_type, reps, K, n_restarts, seed, dm_mode, D)` — VQE completo a un singolo punto: multistart, ottimizzazione a due stadi, calcolo di fidelity e osservabili contro il benchmark esatto. Gli argomenti `dm_mode/D` esistono per compatibilità futura ma restano ai valori di default (nessun DM) in questo documento.
- `vqe_sweep(b_values, J, Jp, ansatz_type, reps, K, n_restarts, seed, verbose)` — chiama `run_vqe` su una griglia di campi b , restituisce la lista dei risultati.
- `_self_test()` — verifica che l'ansatz a parametri nulli riproduca esattamente lo stato di Kambe atteso.
- `_plot_row(axes, ...), plot_grid(all_results, J, base_path)` — costruiscono la figura a 2 righe $\times$ 4 colonne (energia, errore, fidelity, magnetizzazione) e la salvano in `.png/.pdf`.

Come si usa. Come libreria: `from vqe_trimer_ring import run_vqe, vqe_sweep, ...`. Come script standalone: `python3 vqe_trimer_ring.py` esegue il self-test, poi lo sweep completo per HA e PMA ($n_b = 10$, $R = 6$) e salva `vqe_trimer_ring.png/.pdf`. Dipende da `trimer_ring_exact.py` nella stessa cartella.

vqe_trimer_ring_W.py + vqe_trimer_ring_W.png

Variante con il gate $W_{ij}(\theta) = \text{CNOT} \cdot R_y(\theta) \cdot \text{CNOT}$ al posto di RBS — mirror diretto dell'ansatz "originale" del dimero (`vqe_dimer.py`), stessa struttura a branching. 3 parametri a $K = 1$ (contro i 6 della versione RBS: nessun layer finale di R_y indipendenti).

Verifiche. Il file non aveva uno script principale eseguibile (`__main__`) né una figura associata: entrambi mancanti a inizio sessione. Costruito e rieseguito uno script equivalente a quello di `vqe_trimer_ring.py` (stesso punto di lavoro, stesso $R = 6$): fidelity $\mathcal{F} = 1$ esatto ovunque, in diversi punti con errore energetico esattamente nullo (10^{-16} o 0.0).

Conclusione. Colma un buco reale nella produzione: prima di questa sessione, la figura di riferimento per il gate W sull'anello semplicemente non esisteva.

Funzioni interne. Stessa struttura di `vqe_trimer_ring.py`, con queste differenze:

- `w_block(qc, theta, q0, q1)` — il blocco W ($\text{CNOT} \cdot R_y \cdot \text{CNOT}$) al posto di `rbs_block`.
- `make_pma_ansatz(J, Jp, b, K)` — qui non ha il layer finale di R_y indipendenti: solo K blocchi W sui tre bond, $3K$ parametri totali.
- `make_pma_ansatz_forced(branch, K)` — variante con il ramo iniziale ("basso" o "alto") fissato indipendentemente da b , per l'esperimento "cosa succede se si forza un ramo ovunque" (usato nel notebook `_spiegato`, non nello script principale).
- `run_vqe_forced(J, Jp, b, branch, K, n_restarts, seed)` — il VQE con il ramo forzato, invece della selezione automatica.
- Tutte le altre funzioni (`make_ha_ansatz`, `_select_target_block`, `_prepare_initial_state`, `_energy`, `run_vqe`, `_self_test`, `vqe_sweep`, `_plot_row`) hanno lo stesso ruolo della versione RBS.

Come si usa. Come libreria: `from vqe_trimer_ring_W import run_vqe, vqe_sweep, ...`. A differenza di `vqe_trimer_ring.py`, questo file non ha un blocco `__main__`: per riprodurre la figura serve uno script esterno che chiami `_self_test()` seguito da `vqe_sweep` per "HA" e "PMA" e da un plotting equivalente a `plot_grid` (è esattamente ciò che è stato fatto in questa sessione per generare `vqe_trimer_ring_W.png`). Dipende da `trimer_ring_exact.py`.

vqe_trimer_ring_nobbranch.py

Ansatz PMA **senza** branching: stato iniziale fisso (X su un solo qubit, indipendente da J, J', b), la scelta del settore emerge dall'ottimizzazione dei parametri anziché da una preparazione diversa per regime. Stesso conteggio di parametri della versione a branching a $K = 1$ (6), ma verificato **più robusto** al termine DM (nota: questo confronto specifico è materiale della fase con DM, citato qui solo come motivazione storica del file).

Conclusione. A $D = 0$ (lo scope di questo documento) raggiunge $\mathcal{F} = 1$ esatto su tutto l'asse b , incluso b_c degenerare — verificato nella sessione in cui è stato prodotto.

Funzioni interne.

- `rbs_block(qc, phi, q0, q1)` — stesso blocco RBS già validato in `vqe_trimer_ring.py`.
- `make_ha_ansatz(reps)` — l'ansatz hardware-efficient, importato localmente da `qiskit.circuit.library` dentro la funzione stessa.
- `pma_2q_trimer_nobbranch(K)` — l'ansatz senza branching: X fisso su un qubit, poi K cicli di [RBS sui tre bond + R_y indipendenti sui tre qubit], $6K$ parametri.
- `_energy(params, ansatz, hamiltonian, estimator)` — stessa funzione obiettivo degli altri moduli.
- `run_vqe(J, Jp, b, K, n_restarts, seed, dm_mode, D)` — VQE a un punto, interfaccia analoga agli altri moduli ma senza `ansatz_type/reps` (qui c'è solo questo ansatz).
- `vqe_sweep(b_values, J, Jp, K, n_restarts, seed, dm_mode, D, verbose)` — chiama `run_vqe` su una griglia di b .

Come si usa. Come libreria: `from vqe_trimer_ring_nobbranch import run_vqe, vqe_sweep`. Nessun blocco `__main__`, nessuna funzione di self-test dedicata (a differenza degli altri due moduli) — la verifica più diretta è chiamare `run_vqe` a $D = 0$ su un punto qualsiasi e controllare che fidelity sia 1.0. Dipende da `trimer_ring_exact.py`.

vqe_trimer_ring_spiegato.ipynb

Guida interattiva a `vqe_trimer_ring.py`: principio variazionale su matrice 8×8 , blocco RBS, ansatz HA vs PMA, self-test, VQE end-to-end su singolo punto e su griglia ridotta.

Verifiche. Eseguito per intero in questa sessione (33 celle, zero errori): tutti i self-test a precisione macchina.

Considerazioni. Contiene anche una sezione di esplorazione del crollo del PMA sotto DM (§8.3) — fuori dallo scope di questo documento, ma non separata in un file a sé: il notebook è quindi, a rigore, a cavallo fra le due fasi.

vqe_trimer_ring_W_spiegato.ipynb

Guida interattiva a `vqe_trimer_ring_W.py`: mirror sezione per sezione di `vqe_dimer_spiegato.ipynb`, stessa struttura, gate W invece di RBS.

Verifiche. Eseguito per intero in questa sessione (29 celle, zero errori): stato iniziale verificato (overlap = 1.000000), PMA a 3 parametri esatto a $D = 0$.

Considerazioni. Stessa osservazione del notebook precedente: contiene anche una sezione con DM (§7), fuori scope qui.

Conclusione generale

L'anello ha una famiglia completa di file per la fase VQE senza DM (produzione RBS, produzione W , produzione senza branching, due guide interattive), con lo stesso livello di verifica (self-test più esecuzione end-to-end, non solo lettura del codice) già stabilito per il dimero.

Nota. `confronto_ansatz_entangler_trimerio_anello.ipynb` e `analisi_espressivita_PMA_anello.ipynb` non sono descritti in questo documento: il primo, pur costruito su questi stessi ansatz, include anche lo sweep con DM Opzione B ed è quindi materiale della fase VQE con DM; il secondo è interamente un'indagine sotto DM. Entrambi verranno documentati insieme al resto della Fase 5 (VQE con DM), non ancora affrontata in modo sistematico.